

CSCI 135 Assignment – Conditionals

CSCI 135 Assignment – Conditionals

September 13th / 15th, 2023

Due Date: Sunday, 9/17/2022 11:59PM

Points: 30

Topic: Introduction to Conditionals/Selection in Python

The goal of this assignment is to use conditionals in Python. You will be using decision constructs (if statements) in your code. Although we haven't talked much about programming style and readability yet, this is a good time to start practicing making your code organized and readable. Pay attention to your header comment and include comments in-line with your code. Use proper indentation and understandable variable names. Use whitespace to help with readability and logical grouping of sections of code.

Submission:

After following the instructions below, please submit the three Python scripts in Moodle. Upload these as separate files – do not zip them. Be sure each file has a header section including the file name, your name, a description of the program, and other useful information such as example usage.

- Shipping.py
- Lottery.py
- Zodiac.py

Grading:

Each program/script is worth 10 points each for a total of 30 points. For each program, you will be graded according to the following criteria:

	Points
Header Comment	2
Program Written Correctly, Following Direction	4
Program Runs Correctly, Including Correct Output	4
Total	10

Part 1: Shipping.py

The first part of your assignment is to create a program that calculates the cost of shipping a package. Name your program `Shipping.py`. Given the weight of the package, which is passed to your program as a command line argument, calculate and display shipping cost based on the following rules:

- The cost of sending a package is \$15.00 for the first two pounds, and \$5.00 for each pound or fraction thereof over two pounds.
- If the package weighs more than 70 pounds, a \$15.00 excess weight surcharge is added to the cost.
- No package over 100 pounds will be accepted.

Write a program that takes the weight of a package in pounds as a command line argument and computes the cost of mailing the package. Be sure to handle the case of overweight packages.

Here are several test input cases:

For the input 135 your program will print:

The package weighs over 100 pounds and cannot be shipped.

For the input 75 your program would print:

It will cost \$395.0 to ship your package.

For the input 60 your program would print:

It will cost \$305.0 to ship your package.

My program's output isn't the same as yours for the example input. Is this okay?

No. Programming requires attention to detail. You are expected to match the example output exactly, barring an error in the specification. The assignment page is a specification, and you are expected to follow it.

Why do I only get one decimal place in my output when I really want two?

We will look at how to format output later in the semester. For now, we will work with the default output.

How do I declare a constant?

```
MAX_WEIGHT = 100
```

(We have not discussed constants and, technically, the line above does not create a constant in Python. With that said, please use this line in your code to define the maximum weight and then use it subsequently to check against the maximum weight.)

How do I round a floating-point number up to the integer that is greater than or equal to that number?

```
import math
...
roundedUpNumber = math.ceil(originalNumber)
```

(Note from Doug: Think about why you need to do this. The purpose is not to produce a cleaner output. The purpose is to get the calculation right. If you are unclear on this, re-read the specification above. If you are still unclear on it, ask. Hint: This was specified but goes beyond what is shown in the examples.)

I named my program Shipper.py or shipping.py or .zipped it or did some other variation on what was specified. Is this okay?

No. This will break our testing script. This could make our TAs grumpy, and you don't want that. Make sure you name the program correctly.

Upload Shipping.py to Moodle as the first part of your submission for this assignment.

Part 2: Lottery.py

Create a program named Lottery.py that takes two numbers as command line arguments. Both numbers should be integers between 1 and 10, and they represent a lottery pick for a simple two number lottery. Your program shall generate two random integers between 1 and 10 as the actual lottery numbers and shall compare them to the command line input picks. You shall use a nested if statement to determine whether the command line picks are winning numbers. Your program shall implement the following logic:

- First, print what the two randomly generated lottery numbers were.
- Then print what the command line arguments (the user's picks) were.
- If there is no match, print "Sorry, no match."
- If one number matches, print "You matched one number! You win \$100!!"
- If both numbers match, then print "You matched both numbers! Congratulations! You win \$1000!!"

Here is example output where no numbers matched:

```
The lottery numbers were 7 and 3.  
Your picks were 2 and 8.  
Sorry, no match.
```

Here is example output where 1 number matched:

```
The lottery numbers were 4 and 2.  
Your picks were 2 and 8.  
You matched one number! You win $100!!
```

Here is example output where both numbers matched:

```
The lottery numbers were 2 and 6.  
Your picks were 6 and 2.  
You matched both numbers! Congratulations! You win $1000!!
```

How do I generate a random integer between 1 and 10?

Use the following code to get a random integer in that range:

```
import random
...
lotteryNumber1 = random.randint(1, 10)
```

(Note from Doug: This approach is slightly different than what was shown in class for a random coin toss and is easier because it directly generates integers. Use it.)

Sometimes I get two random numbers that are the same. Is this OK?

Yes, for this simple lottery, it is possible to get the same number twice.

The random numbers that I get are not the same as in your examples. Is this OK?

Yes, this is likely to happen. In fact, it would be strange if your randomly generated numbers are the same as the examples unless the same random seed was used.

(Note from Doug: Do not set the seed for the random number generator as part of your program. When no seed is specified, Python will use an OS-specific method to seed the pseudo-random number generator. It may be the system time, but it could be something else.)

Upload Lottery.py to Moodle as the second part of your submission for this assignment.

Part 3: Zodiac.py

Create a program `Zodiac.py` that determines the Chinese zodiac sign given a year of birth as a command line argument. The year of birth should be an integer. Use if statements in your program to determine which zodiac sign to output.

The rules for the Chinese zodiac signs are:

- It is based on a 12-year cycle.
- If you were born in a year that is a multiple of 12, your sign is `Monkey`.
- If your year of birth, when divided by 12, has a remainder of 1, your sign is `Rooster`.
- The possibilities corresponding to each remainder are:

```
0: Monkey
1: Rooster
2: Dog
3: Pig
4: Rat
5: Ox
6: Tiger
7: Rabbit
8: Dragon
9: Snake
10: Horse
11: Sheep
```

For the input 1975 your program would print:

```
You were born in the year of the Rabbit.
```

Upload `Zodiac.py` to Moodle as the third part of your submission for this assignment.

EXTENSIONS

You will not submit the answers to these extensions as part of your assignment submission. You should work through and understand them to learn more and practice. You may be held responsible for the content within, so you really should work through them ...

From the Liang book - NOTE: We will **not** use MyProgrammingLab in the book. You should be able to do exercises from Liang without MyProgrammingLab.

1. *Revisit GuessBirthday.py in Listing 4.3. (It was listed as an extension in the last lab.) Try it out. How and why does the binary representation of a number help in making this program work? How are conditionals used? Notice Liang's use of eval() and recall that I have said we should not use that function. Why? What should be done instead?*
2. *Notice the flowcharts that Liang uses to show conditional logic. See Figures 4.1, 4.3, and 4.5. Is actual Python code always shown in these diagrams or something else?*
3. *Work through other exercises and examples in Chapter 4. Do the easy ones first. Try some of the more challenging ones.*

From the Things to Do section at the end of Chapter Four of our optional Lubanovic book:

- 4.1 *Choose a number between 1 and 10 and assign it to the variable secret. Then, select another number between 1 and 10 and assign it to the variable guess. Next, write the conditional tests (if, else, and elif) to print the string 'too low' if guess is less than secret, 'too high' if greater than secret, and 'just right' if equal to secret.*
- 4.2 *Assign True or False to the variables small and green. Write some if/else statements to print which of these matches those choices: cherry, pea, watermelon, pumpkin.*

From me:

1. *The Lottery.py problem allows the two random lottery numbers to be the same. What would it take to ensure they are different? Can you accomplish this with constructs we have covered so far in class, or do you need something more?*
2. *True or False: The elif construct in Python is nice to have but is not necessary since you can implement the same logic using if and else statements alone. Explain.*